

Fake vs. Real Restaurant Review Classification Using Linguistic Features

Yasaman Ebrahimi, Bao Hong Gia Nguyen

Abstract—This project analyzed whether ten required linguistic features can be used to classify restaurant reviews as fake or real. The dataset contained 260 reviews, with 130 fake reviews and 130 real reviews. We extracted average word length, average sentence length, number of words, number of verbs, number of adjectives, passive voice count, number of sentences, content diversity, number of typos, and typo ratio. The data were shuffled and split into 60% training, 20% validation, and 20% test sets while keeping the two classes balanced. We trained Logistic Regression, Random Forest, and Support Vector Machine (SVM) models using ANOVA feature selection, 5-fold cross-validation, and hyperparameter tuning. Random Forest had the highest test performance, with 73.08% accuracy and a 69.57% F1-score. Overall, the results show that the ten features contain useful information for separating fake and real reviews, but they do not explain the difference completely. Number of adjectives was the highest-ranked feature in all three final models.

Index Terms—fake review detection, text classification, feature extraction, machine learning, random forest, support vector machine

I. INTRODUCTION

Fake online reviews can make it harder for customers to judge a restaurant using online feedback. The main goal of this project was to test whether simple linguistic characteristics in a review are related to whether the review is fake or real. Instead of training directly on the review text, the project required us to extract exactly ten numerical features and use only those features for the classification models.

The project focused on two main questions. First, we wanted to know whether the ten required linguistic features contain enough information to classify a review as fake or real. Second, we wanted to know which of the ten features appear to have the strongest relationship with the classification result. We used a balanced dataset with 260 restaurant reviews, including 130 fake reviews and 130 real reviews, so the two classes had the same number of examples.

To answer these questions, we used the same general process for all three models. We cleaned the review text, extracted the ten required features once, created balanced train, validation, and test splits, performed feature selection, tuned each model with 5-fold cross-validation, retrained the best version using the combined training and validation data, and then evaluated the final models on the test set. Using the same split and feature data for all three models made the model comparison more consistent.

II. DATASET AND PREPROCESSING

A. Dataset

The data came from a spreadsheet containing the restaurant name, review text, a fake/real label, and a sentiment label. After loading the spreadsheet, we renamed the columns to shorter names so they were easier to use in Python. The sentiment label was kept in the original data for reference, but it was not used for model training because the project required us to train only with the ten extracted linguistic features. Rows with missing review text or missing fake/real labels were removed, the review column was converted to a string, and the fake/real label was converted to an integer.

After preprocessing, the dataset contained 260 usable reviews. The classes were exactly balanced, with 130 reviews labeled 0 (fake) and 130 reviews labeled 1 (real). This made the model evaluation easier to interpret because accuracy and F1-score were not being affected by one class having many more examples than the other.

B. Text Processing and Feature Extraction

The feature extraction process was designed so that the same ten features could be calculated once and then reused for every model. The final feature data were saved as `reviewFeatures.xlsx`. Alphabetic words were identified with a regular expression so punctuation would not increase the word count. NLTK sentence tokenization was used to count sentences and calculate average sentence length. We used the spaCy `en_core_web_sm` model for part-of-speech and dependency information, NLTK stopwords for content diversity, and PySpellChecker for typo detection. Table I summarizes the ten features used in the project.

For content diversity, punctuation was already removed by the alphabetic-word extraction. The words were converted to lowercase and English stopwords were removed before the ratio was calculated:

$$CDV = \frac{|V|}{N_c}, \quad (1)$$

where $|V|$ is the number of unique content words and N_c is the total number of content-word tokens. The typo ratio was calculated as

$$TPR = \frac{NTP}{NWO}. \quad (2)$$

The saved feature file contains only the ten required features and the label column. This allowed us to reuse the exact same extracted features for all three models instead of running the text-processing step again for every model.

TABLE I
EXTRACTED LINGUISTIC FEATURES

Feature	Definition used in the project
AWL	Average number of characters in the alphabetic words.
ASL	Average number of alphabetic words in each detected sentence.
NWO	Total number of alphabetic words in the review.
NVB	Number of spaCy tokens tagged as verbs.
NAJ	Number of spaCy tokens tagged as adjectives.
NPV	Number of passive-voice constructions based on passive subject and auxiliary dependencies.
NST	Total number of sentences detected by NLTK.
CDV	Number of unique non-stopword content words divided by the total number of content words.
NTP	Number of unknown alphabetic words with at least three characters reported by the spelling checker.
TPR	Number of typos divided by the total number of words.

III. DATA SPLIT AND MODEL TRAINING

A. Balanced 60:20:20 Split

The data were split using stratified shuffled splitting with a fixed random seed of 42. The first split created 156 training examples and 104 temporary examples. The temporary data were then split evenly into validation and test sets. This gave us 156 training reviews, 52 validation reviews, and 52 test reviews, which matches the required 60%:20%:20% split.

Each part of the data stayed exactly balanced. The training set contained 78 fake and 78 real reviews. The validation set contained 26 fake and 26 real reviews, and the test set also contained 26 fake and 26 real reviews. For model tuning, we combined the training and validation sets into a 208-review train+validation set with 104 fake and 104 real reviews. This combined set was shuffled again, while the 52-review test set was kept separate until the final evaluation.

B. Feature Selection and Cross-Validation

Each model used `SelectKBest` with the ANOVA F-test (`f_classif`) as the feature-selection method. Feature selection was included inside each model pipeline so that it was repeated correctly during cross-validation instead of being fitted once using all of the train+validation data. The number of selected features, k , was also treated as a tuning parameter with values 3, 5, 7, 9, and 10.

We used stratified 5-fold cross-validation with shuffling and a random seed of 42. Grid search used F1-score as the main optimization metric because F1 was one of the main required evaluation measures for the project. After the best parameters were selected, `GridSearchCV` refit that model using the full train+validation set.

C. Classification Models and Hyperparameters

We trained three different classification models so we could compare a linear method, a tree-based method, and an SVM under the same conditions. Logistic Regression was used as the linear baseline and its inputs were standardized because the model is affected by feature scale. Random Forest was used as a nonlinear tree ensemble and did not require scaling.

SVM was also standardized and was tested with both linear and radial-basis-function kernels.

Table II shows the tuning ranges and the best parameter combination for each model. The best cross-validation F1-scores were 0.6116 for Logistic Regression, 0.6297 for Random Forest, and 0.6052 for SVM.

The best fitted version of each model was saved using `Joblib` as `best_logistic_regression.joblib`, `best_random_forest.joblib`, and `best_svm.joblib`.

IV. EVALUATION METRICS

We evaluated the final models using both the train+validation data and the untouched test set. Accuracy measures the percentage of all predictions that are correct:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (3)$$

Precision and recall for the positive class are

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad (4)$$

and F1-score is the harmonic mean of precision and recall:

$$F_1 = 2 \frac{PR}{P + R}. \quad (5)$$

F1-score was used as the main metric during model tuning and final comparison. We also recorded how many features each final model selected and which feature ranked highest. Random Forest feature importance came from the fitted tree ensemble. For Logistic Regression and the linear SVM, the features were ranked using the absolute coefficient sizes after scaling.

V. RESULTS

Table III shows the final performance of the three models. Random Forest had the highest test result, with 73.08% accuracy and a 69.57% F1-score. Logistic Regression and SVM both reached 69.23% test accuracy, while their test F1-scores were 61.90% and 63.64%, respectively.

Figure 1 compares the train+validation and test results. The Random Forest has a much larger gap between the two sets than the other models. Its accuracy dropped from 84.13% on train+validation to 73.08% on the test set, while its F1-score dropped from 82.54% to 69.57%. This suggests that the Random Forest learned some patterns in the training data that did not generalize as well to the test data. Logistic Regression and SVM had lower train+validation scores, but their train-to-test gaps were smaller, which suggests less overfitting.

The feature rankings showed a similar pattern across the models. NAJ, which is the number of adjectives, was the highest-ranked feature in all three final models. For Logistic Regression, the ranking was NAJ, NST, NTP, TPR, CDV, NWO, NPV, ASL, and NVB. For Random Forest, the ranking was NAJ, NWO, NVB, NST, and NTP. For SVM, the ranking was NAJ, NST, NTP, NWO, TPR, NPV, NVB, CDV, and ASL.

TABLE II
HYPERPARAMETER TUNING AND BEST CROSS-VALIDATION CONFIGURATION

Model	Parameters searched	Best parameters	CV F1
Logistic Regression	$k \in \{3, 5, 7, 9, 10\}$; $C \in \{0.01, 0.1, 1, 10, 100\}$	$k = 9, C = 1$	0.6116
Random Forest	$k \in \{3, 5, 7, 9, 10\}$; estimators $\in \{100, 200\}$; max depth $\in \{\text{None}, 5, 10\}$; min split $\in \{2, 5\}$	$k = 5, 100$ estimators, max depth = 5, min split = 5	0.6297
SVM	$k \in \{3, 5, 7, 9, 10\}$; $C \in \{0.01, 0.1, 1, 10, 100\}$; kernel $\in \{\text{linear}, \text{RBF}\}$; gamma $\in \{\text{scale}, \text{auto}\}$	$k = 9, C = 10$, linear kernel, gamma=scale	0.6052

TABLE III
MODEL PERFORMANCE COMPARISON

Model	Train+Val Acc.	Train+Val F1	Test Acc.	Test F1	# Features	Top Feature
Logistic Regression	62.50%	60.20%	69.23%	61.90%	9	NAJ
Random Forest	84.13%	82.54%	73.08%	69.57%	5	NAJ
SVM	63.94%	61.14%	69.23%	63.64%	9	NAJ

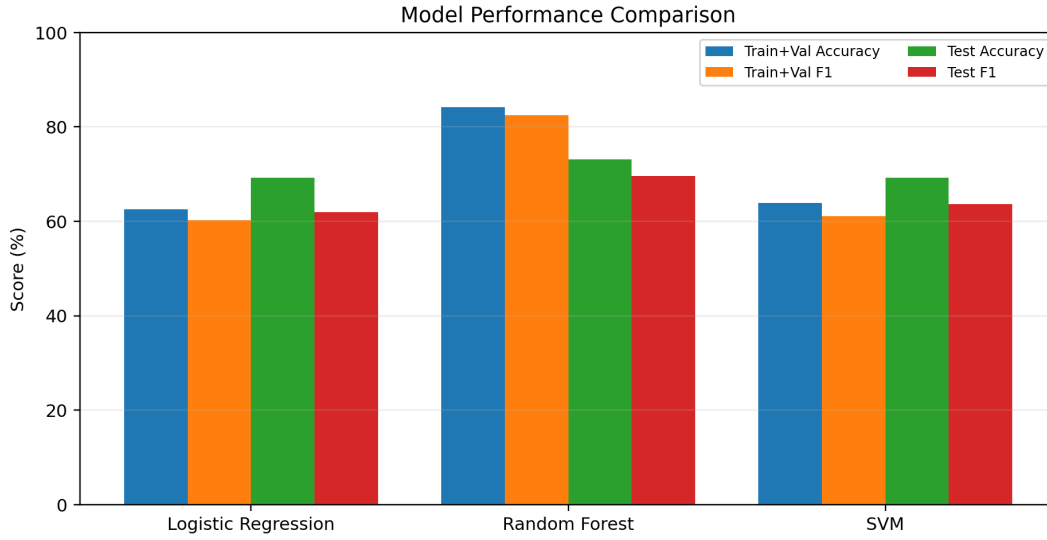


Fig. 1. Training-plus-validation and test accuracy/F1 for the three final models.

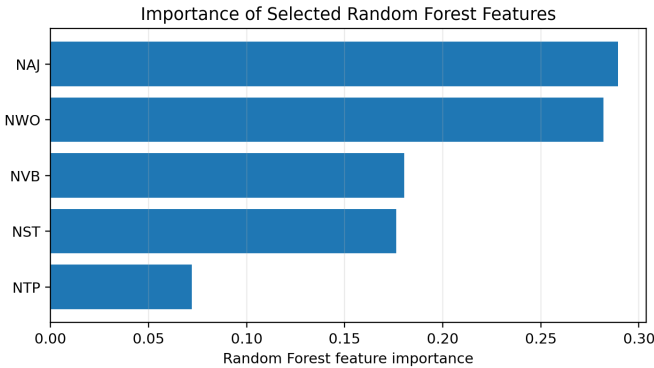


Fig. 2. Feature importance for the five features selected by the final Random Forest model.

Figure 2 shows the relative importance of the five features selected by Random Forest.

The project benchmark listed 79.09% accuracy and a 76.99% F1-score. Our highest test result came from Random Forest, which was 6.01 percentage points below the benchmark accuracy and 7.42 percentage points below the benchmark F1-score. This means the current implementation did not reach the benchmark. We kept the actual held-out test results in the report instead of replacing them with the higher train+validation scores.

VI. ANALYSIS AND DISCUSSION

Overall, the results show that the ten linguistic features can be used to predict whether a review is fake or real to some extent, but the prediction strength is still limited. The highest test accuracy was about 73%, so the models were finding useful patterns, but a noticeable number of reviews were still classified incorrectly. Since the dataset was balanced between fake and real reviews, the result is better than a 50% random

guess, but there is still a large amount of information that the ten features are not capturing.

The most noticeable feature result was the number of adjectives. NAJ ranked first for Logistic Regression, Random Forest, and SVM. This suggests that the amount of descriptive language may be related to whether a review is fake or real in this dataset. Sentence count and typo-related features also appeared near the top for more than one model. However, these rankings only show association inside the fitted models. They do not prove that using more adjectives, having more typos, or changing sentence count directly makes a review fake or real.

Random Forest had the highest test scores, but it also showed the clearest sign of overfitting. Its train+validation F1-score was 82.54%, while its test F1-score was 69.57%, which is a difference of about 13 percentage points. The cross-validation F1-score of 0.6297 was much closer to the test result than the 0.8254 train+validation result. This supports the idea that the cross-validation score gives a more realistic estimate of how the model will perform on unseen reviews than the score calculated on the same data used to fit the final model.

The two linear models were more consistent between train+validation and test data. Logistic Regression and the linear SVM both selected nine of the ten features and both had 69.23% test accuracy. Their test F1-scores were slightly higher than their train+validation F1-scores, so there is not the same clear overfitting pattern seen in Random Forest. That being said, their overall scores were lower, which suggests that the ten available features do not create a very strong linear separation between fake and real reviews.

There are also several limitations in the feature extraction. The spelling checker can count uncommon restaurant names or uncommon words as typos even when they are not actual spelling mistakes. Passive voice depends on dependency parsing, so its count can change if the parser does not analyze a sentence correctly. Content diversity can also be affected by review length because short reviews can naturally have a high percentage of unique words. These issues can add noise to the feature data, and the effect is more noticeable because the dataset contains only 260 reviews.

VII. CONCLUSION

This project analyzed fake and real restaurant reviews using ten required linguistic features. We cleaned the source data, extracted the features once, saved the feature file, created balanced and shuffled 60%:20%:20% splits, used ANOVA F-test feature selection, ran 5-fold stratified cross-validation, tuned three classifiers, saved the best version of each model, and evaluated the final models on both train+validation and test data.

The main conclusion is that the required features can classify fake and real reviews to a limited extent, but the models did not reach the listed benchmark. Random Forest had the highest held-out test performance, with 73.08% accuracy and a 69.57% F1-score, but it also had the largest gap between training and test performance. Logistic Regression and SVM

were more consistent but had lower F1-scores. Across all three models, the number of adjectives was the highest-ranked feature.

In the future, the project could be improved by testing other feature-selection methods, using more careful typo handling, applying stronger regularization, or expanding the hyperparameter search while still using only the ten permitted features. A larger review dataset would also make the model comparison more stable and would make it easier to tell whether the feature patterns found here continue to hold on new data.

GROUP CONTRIBUTION

The final notebook includes the preprocessing, feature extraction, data splitting, model tuning, evaluation, and saved outputs used for the report.

SUBMISSION NOTE

Overleaf project link:

<https://www.overleaf.com/read/yycfrcqwtcwp#a60ada>